Team: Background Checkers
Team members: Anthony Man, Amari Thomas, Braeden Alonge, Max Schemenauer
Assignment: System Design and Architecture
Course: CSC 5870 - LLM Post-Training & Retrieval
Date: 10/13/2026

LLM post-training, retrieval, and/or agentic design methods and design rationale

Our design requires the model to do four things: 1) extract individual claims of fact from the input text; 2) make a structured tool request to search the internet for information related to each point; 3) summarize/extract the information from the tool's return; and 4) classify each point as "Supported," "Contradicted," or "Insufficient Evidence" based on the internet findings. Our model will need to be fine-tuned for each of these. We plan to use LoRA for our fine-tuning.

The system will use a web-based RAG method, which is necessary because it allows it to retrieve knowledge from the internet rather than relying on internal knowledge stored in the model. Without using this approach, the model would have to rely on facts it learned through its training, which may be outdated or even outright wrong.

Because our design calls for output to be generated in a structured manner, simply prompting the baseline model may not be enough, which is why we'll need to perform post-training. For both summaries (one for the text and the other for the tool response), as well as the fact matching, each assertion must be written in a clean format (such as JSON) to facilitate the handling of the information. Likewise, the internet tool call will require the LLM to output a JSON in the specific format that the tool needs. Without post-training, the model might not be able to match our required schema out of the box.

While both web-based RAG, structured tool calls, JSON-formatted output, and LoRA fine-tuning are well-established methods, our contribution is the combination of all of them for the purpose of a fact-checking pipeline.

Functional components and interactions

The system accepts an English passage, such as an LLM-generated response, through a text input. A claim extractor reads the passage and separates its factual assertions into individual claims that can be checked separately. Opinions and conversational text are left out. Then, each claim is passed to a search planner, which creates a web query that preserves the claim's meaning.

The search planner sends each query to an internet search service. An evidence selector then reviews the returned pages, identifies information relevant to the claim, and records each source's title and URL. When enough relevant pages are available, it attempts to gather the evidence from at least three independent sources. If the first search is too broad or produces little useful information, the planner will resend a new query that is more specific.

A claim evaluator compares the gathered evidence with the original claim and assigns one of three results: "Supported," "Contradicted," or "Insufficient Evidence." The third result will derive from missing pages, unclear findings, not enough evidence, or conflicting sources. The produced result will include the label and links to the sources used so that the users can look at the evidence themselves.

These components check claims separately instead of looking at the passage as one idea to analyze. They also use multiple (at least three) sources when possible, which ensures that our model does not choose the first evidence it finds to use as the sole truth evaluation. A browser extension could eventually provide a convenient way to submit text, but the verification process will be able to operate through a basic text interface without needing a browser extension (using pasted text).

Architecture diagram and explanation

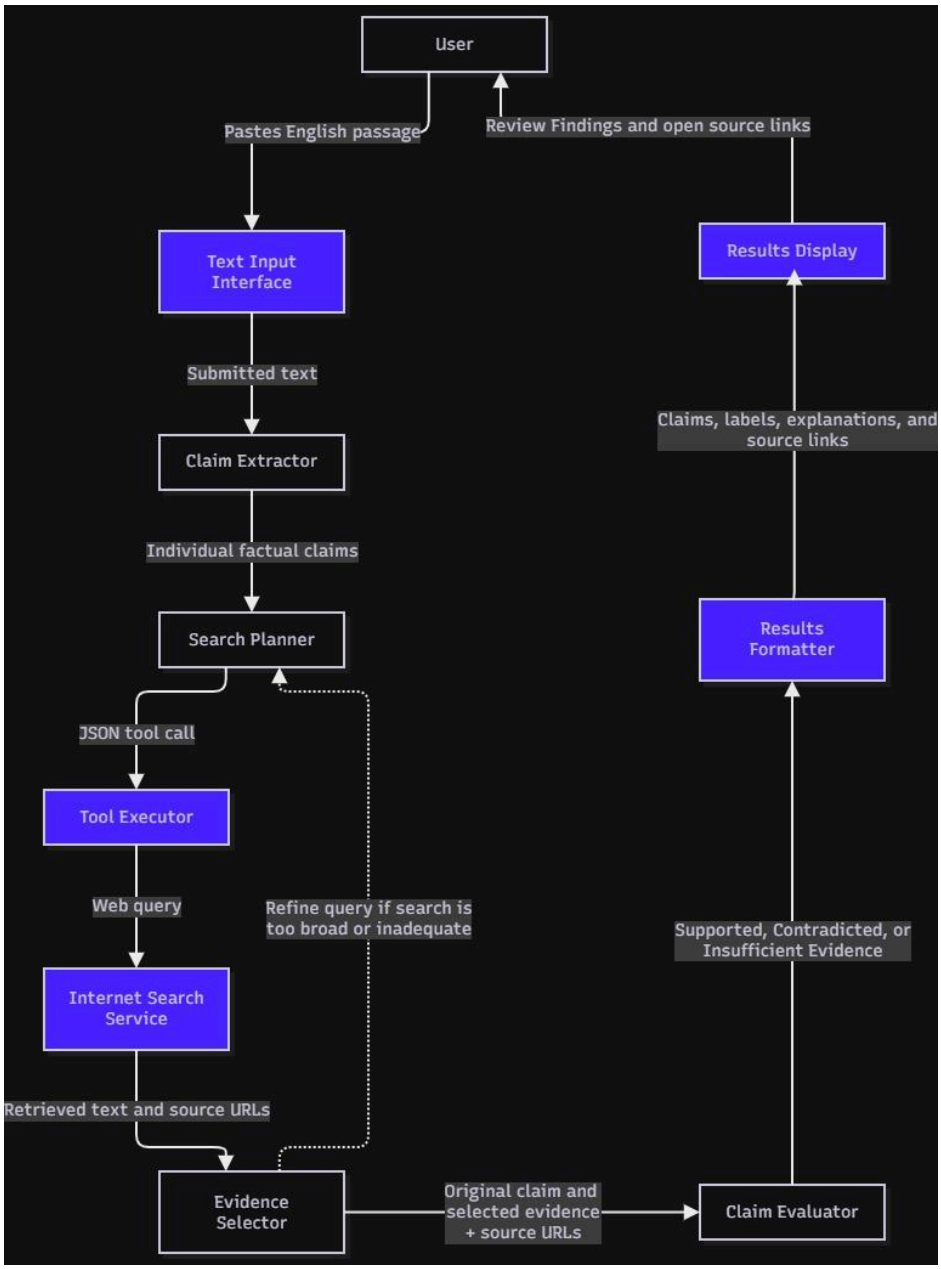


Figure 1. Fact-checking diagram showing components described in the "Functional components and interactions" section. Outlined boxes are LoRA-tuned model components; blue boxes are non-model components (code or external services). Solid arrows show the main flow of information from user input to the displayed result. The dotted arrow shows a query-refinement loop when initial results are too broad or inadequate.

Component Descriptions (Model components)

- Claim Extractor: Separates the passage into individual factual claims and leaves out opinions
- Search Planner: Turns each claim into a JSON tool call containing a web query that preserves the claim's meaning
- Evidence Selector: Picks out the information relevant to the claim from the retrieved pages. Records the source title and URL
- Claim Evaluator: Compares the evidence with the original claim and assigns Supported, Contradicted, or Insufficient Evidence

Component Descriptions (Non-model components)

- Text Input Interface: Accepts user input
- Tool Executor: Validates the JSON produced by the model, calls the search service, and returns the results
- Internet Search Service: An external service that retrieves current articles and pages from the web
- Results Formatter: Arranges labels, explanation, and source links to be presented to the user
- Results Display: Shows the labels, explanation, and source links so the user can open the sources and check the evidence

Component Descriptions (Refinement loop)

- Refinement loop: If the first search is too broad or produces little useful information, the Evidence Selector sends the request back to the Search Planner, which resends a more specific query.

Knowledge and data representation

This system will primarily use structured JSON schemas to pass information between its components in order to maintain state and context. The first stage of data transformation is the input and claim representation. The initial input is simply an unstructured English passage that should be at least a few hundred words. This passage is transformed by an LLM into a list of structured individual fact claims. Each claim is represented as a JSON object with a unique identifier and the factual statement. Additionally, there can be a context section with a very brief description to resolve nuance about the context this claim was made in.

For each of these extracted claims, the search planner will design specific web queries that are designed to preserve the original meaning of the claim, in context. This is formatted as a JSON tool call to be used by an external search API.

The evidence selector parses the search tool's unstructured output (likely HTML or plain text) and extracts the relevant information. This information will be structured as an evidence array, which stores the summarized findings alongside metadata like the source's title and its URL. The system requires this array to have evidence from at least three distinct web pages when sufficient sources are available. The evidence selector also prioritizes certain domains like .gov or peer-reviewed academic publications.

The claim evaluator looks at the original claim object alongside the gathered evidence array. It then compares the original claim to the evidence and assigns a label: "Supported," "Contradicted," or "Insufficient Evidence" based on the internet findings. The final data structure delivered to the user/front-end is a payload containing the original claim situated in context, the label, and the sources with URLs.

A large limitation of this pipeline is the assumption that online consensus is a good proxy for factual truth. This is a known limitation, and the user should be made aware that the labels pertain to internet consensus, not objective truth, which is not a trivial retrieval problem. Additionally, this system does not rely on a persistent database, which is fine for current information, but can be a hindrance for well-established claims, where we are missing a curated set of reliable sources. Another data-related concern is the slippery slope of having LLMs decide at every step of the process. They first have to decide whether something is an opinion or claim, then what context it was in, then how to search for it, and so on. This can heavily dilute the original nuance, making the final product significantly less reliable.

AI Usage Disclaimer

ChatGPT was used to help further understand the current techniques with respect to RAG and tool usage. It was also used to help workshop ideas for what techniques we might use at each step in our pipeline, as well as to check the quality of the write-up.